

Python Programming Foundations

5-Day Beginner Python Course

Table of Contents (Facilitator)

- 1. [Facilitator Preparation](#)
- 2. [Day 1: Python Basics - Facilitator Guide](#)
- 3. [Day 2: Python Expressions, Statements & Conditions - Facilitator Guide](#)
- 4. [Day 3: Python Loops, Lists & Data Structures - Facilitator Guide](#)
- 5. [Day 4: Python Functions, Scope & Errors - Facilitator Guide](#)
- 6. [Day 5: Python Input/Output, Algorithms & Complete Programs - Facilitator Guide](#)
- 7. [Appendix: Answer Keys](#)
- 8. [Appendix: Python Setup Guide](#)

Facilitator Preparation

Master Props Checklist (Python Edition)

Item	Quantity	Purpose	Day Used
Computers with Python installed	1 per participant	Coding exercises	All
VS Code or IDLE	1 per computer	Code editor	All
Projector/Display	1	Code demonstrations	All
Whiteboard + markers	1 large	Diagrams and code	All
Printed worksheets	1 set per participant	Exercises	All
Index cards	100	Variable exercises	Day 1
Sticky notes	10 pads	Code structure	Day 5
Timer	1	Activity timing	All
Python cheat sheets	1 per participant	Reference	All
USB drives	5	Backup solutions	All

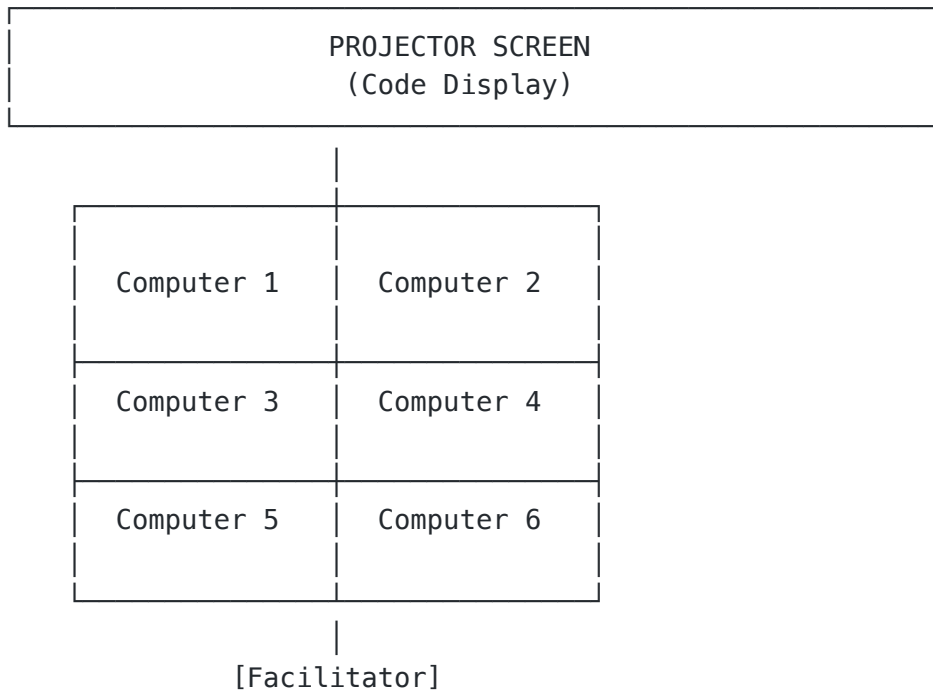
Python Installation Checklist

Before Day 1, ensure:

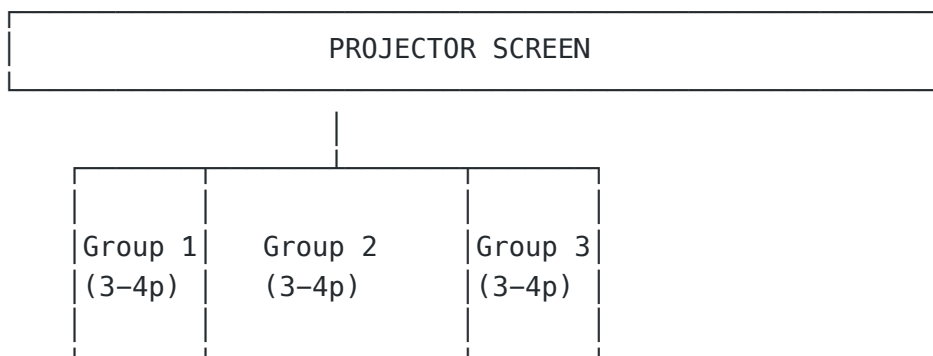
- [] Python 3.6+ installed on all computers
- [] VS Code or IDLE installed
- [] Python extension installed in VS Code
- [] Test that `print("Hello")` works
- [] Test that `input()` works
- [] Test that file operations work (`open()`)
- [] Internet access (for additional resources)

Classroom Setup Guide

Day 1-2: Computer Lab Configuration



Day 3-5: Group Configuration



Day 1: Python Facilitator Guide

Python Basics - Data Types, Variables & Operators

Session Plan Overview

Time	Activity	Duration
09:00-09:15	Opening & Python Installation Check	15 min
09:15-09:45	Python Data Types Theory	30 min
09:45-10:15	EXERCISE: Identify Python Types	30 min
10:15-10:45	Python Variables Theory	30 min
10:45-11:15	EXERCISE: Python Variable Lockers	30 min
11:15-11:45	Python Operators Theory	30 min
11:45-12:15	EXERCISE: Python Calculator Game	30 min
12:15-13:00	LUNCH	45 min
13:00-14:00	EXERCISE: Spot the TypeError	60 min
14:00-15:00	BUILD: Complete Python Data Map	60 min
15:00-15:30	Day 1 Wrap-Up	30 min

Morning Session Script

09:00 - 09:15 | Opening & Python Setup Check

[SAY] "Good morning everyone. Welcome to Python Programming Foundations. Before we start, let's make sure everyone is set up."

[CHECKLIST - Go through each item]

1. Open Terminal/Command Prompt:
- Windows: Press Windows key, type "cmd"

- Mac: Press Cmd + Space, type "terminal"

2. Check Python Installation:

```
python --version
```

or

```
python3 --version
```

Expected output: Python 3.x.x

3. Open VS Code or IDLE:

- VS Code: Open from Start Menu / Applications
- IDLE: Comes with Python

4. Run a test script:

```
print("Hello Python!")
```

[SAY] "If anyone is having issues, raise your hand and I'll help. Once we're all set up, let's begin."

09:15 - 09:45 | Python Data Types Theory

[SAY] "Today we're going to learn the building blocks of Python programming. Let's start with data types."

[WRITE ON WHITEBOARD]

Today's Goal: Understand WHAT data is
and WHAT we can DO with it in Python.

[SAY] "Think of data types like different containers. Numbers are like jars. Text is like folders. Lists are like filing cabinets. Each holds a different kind of thing."

[PRESENT - Use Projector/Whiteboard]

Python's 7 Core Data Types:

1. **int** (Integer): Whole numbers

25, -10, 1000, 0

2. **float** (Floating Point): Numbers with decimals

3.14, -0.5, 100.0

3. **str** (String): Text

"Hello", 'John', "123" # quotes make it text!

4. **bool** (Boolean): True/False

True, False

5. **list** (List): Ordered collection

[1, 2, 3], ["apple", "banana"]

6. **dict** (Dictionary): Key-value pairs

{"name": "Alice", "age": 25}

7. **None** (NoneType): Nothing/Empty

None

[**SAY - Pointing to each**] "Let's look at each one with examples you'll use every day."

[**DEMONSTRATE LIVE**] Open Python interpreter or Jupyter notebook and run:

```
# Numbers
age = 25
price = 19.99
print(type(age))    # <class 'int'>
print(type(price))  # <class 'float'>

# Text
name = "Alice"
print(type(name))   # <class 'str'>

# Boolean
is_student = True
```

```
print(type(is_student)) # <class 'bool'>

# List
scores = [85, 90, 78]
print(type(scores)) # <class 'list'>

# Dictionary
person = {"name": "Alice", "age": 25}
print(type(person)) # <class 'dict'>

# None
nothing = None
print(type(nothing)) # <class 'NoneType'>
```

[SAY] "Notice the `type()` function. It tells you what type you're working with. You'll use this a lot when debugging."

09:45 - 10:15 | EXERCISE: Identify Python Types

[SAY] "Now let's practice. I'm going to show you values, and you'll tell me their Python type."

[PROJECT/WRITE - One at a time]

Value	Python Type	Check with <code>type()</code>
25	int	<code>type(25) → <class 'int'></code>
"Hello"	str	<code>type("Hello") → <class 'str'></code>
True	bool	<code>type(True) → <class 'bool'></code>
[1, 2, 3]	list	<code>type([1,2,3]) → <class 'list'></code>
None	NoneType	<code>type(None) → <class 'NoneType'></code>
3.14	float	<code>type(3.14) → <class 'float'></code>
{"name": "Alice"}	dict	<code>type({"name": "Alice"}) → <class 'dict'></code>
"123"	str	<code>type("123") → <class 'str'></code>

[SAY] "Good. Now open Python on your computer and verify each one."

[GIVE 5 MINUTES]

[DEBRIEF] "Any surprises? Yes, "123" is a string, not a number. Quotes make it text!"

10:15 - 10:45 | Python Variables Theory

[SAY] "Now let's talk about VARIABLES. In Python, a variable is like a label you stick on a box."

[WRITE ON WHITEBOARD]

Variable = A NAME that points to a VALUE

Syntax:

```
variable_name = value
```

Examples:

```
age = 25
```

```
name = "John"
```

```
is_student = True
```

```
scores = [85, 90, 78]
```

```
person = {"name": "Alice", "age": 30}
```

[SAY] "Variable naming rules in Python:"

[WRITE ON WHITEBOARD]

1. Must start with a letter **or** underscore (**_**)
2. Can contain letters, numbers, underscores
3. Case-sensitive (age $\neq$ Age)
4. Cannot use Python keywords (**if**, **for**, **while**)

Good variable names:

```
user_age  
first_name  
total_score  
is_active  
item_list
```

Bad variable names:

```
2name      # Can't start with number  
user-name  # No hyphens  
if         # Reserved keyword  
class      # Reserved keyword
```

[SAY] "Important: In functional programming, we NEVER change variables. Instead, we create NEW variables."

[DEMONSTRATE]

```
# WRONG (mutation)
age = 25
age = 26 # Bad practice in FP

# RIGHT (immutable)
age = 25
new_age = 26 # Create a new variable
```

[SAY] "Why? Because if I gave you my locker and said 'age is 25', and then later said 'age is 26', you'd be confused. Which age? The old one? The new one? By keeping both, we have a perfect history."

10:45 - 11:15 | EXERCISE: Python Variable Lockers

[SAY] "Let's practice Python variables."

[DISTRIBUTE] Give each participant index cards and sticky notes.

[SAY] "Open your Python interpreter and create these variables:"

[CALL OUT]

```
age = 25
first_name = "John"
is_active = True
scores = [85, 90, 78]
person = {"name": "Alice", "age": 30}
```

[SAY] "Now, change age from 25 to 26. BUT you cannot change the original variable."

Expected output:

```
age = 25
new_age = 26 # Create a new variable
```

[SAY] "Good! Now practice using `type()` on each variable."

```
print(type(age))           # <class 'int'>
print(type(first_name))    # <class 'str'>
print(type(is_active))     # <class 'bool'>
print(type(scores))        # <class 'list'>
print(type(person))        # <class 'dict'>
```

Common Mistakes to Watch For:

- Using `=` instead of `==` in conditions
- Forgetting quotes for strings
- Using reserved keywords as variable names
- Case sensitivity issues

11:15 - 11:45 | Python Operators Theory

[SAY] "Now we've got data and names for data. What can we DO with it? OPERATORS!"

[PROJECT/WRITE - Permanent Reference]

Python Operators Table

Type	Operators	Example	Result
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>//</code> <code>%</code> <code>**</code>	<code>5 + 3</code>	<code>8</code>
Comparison	<code>==</code> <code>!=</code> <code>></code> <code><</code> <code>>=</code> <code><=</code>	<code>10 > 5</code>	<code>True</code>
Logical	<code>and</code> <code>or</code> <code>not</code>	<code>True and False</code>	<code>False</code>
String	<code>+</code> <code>*</code> <code>in</code>	<code>"Hello" + "!"</code>	<code>"Hello!"</code>
List	<code>+</code> <code>*</code> <code>in</code>	<code>[1,2] + [3,4]</code>	<code>[1,2,3,4]</code>

[DEMONSTRATE LIVE]

```
# Arithmetic
print(5 + 3)      # 8
print(20 / 5)     # 4.0 (float)
print(17 // 5)    # 3 (floor division)
print(17 % 5)     # 2 (remainder)
print(2 ** 3)     # 8 (exponent)

# Comparison
print(10 > 5)     # True
print(5 == "5")  # False (different types)

# Logical
print(True and False) # False
print(True or False)  # True
print(not True)       # False

# String
print("Hello" + " " + "World") # "Hello World"
print("Ha" * 3)             # "HaHaHa"
print("a" in "apple")       # True
```

```
# List
print([1,2] + [3,4]) # [1,2,3,4]
print([0] * 5)       # [0,0,0,0,0]
print(2 in [1,2,3]) # True
```

[SAY - Key Points]

- `/` always returns a float
- `//` returns an integer (floor division)
- `==` checks value only
- `in` checks membership
- `and` requires both True
- `or` requires at least one True

11:45 - 12:15 | EXERCISE: Python Calculator Game

[SAY] "Let's practice with Python. Open your Python interpreter and solve these."

[PROJECT/WRITE]

Expression	Python Code	Expected Result
$5 + 3 \times 2$	<code>5 + 3 * 2</code>	11
<code>"Hello" + " " + "World"</code>	<code>"Hello" + " " + "World"</code>	"Hello World"
<code>10 > 5 and True</code>	<code>10 > 5 and True</code>	True
<code>[1,2] + [3,4]</code>	<code>[1,2] + [3,4]</code>	[1,2,3,4]
<code>15 % 4</code>	<code>15 % 4</code>	3
<code>"Apple".upper()</code>	<code>"Apple".upper()</code>	"APPLE"
<code>len("Hello")</code>	<code>len("Hello")</code>	5
$(5 + 5) * (2 + 3)$	<code>(5 + 5) * (2 + 3)</code>	50

[GIVE 10 MINUTES]

[DEBRIEF] Go through each answer:

- $5 + 3 * 2 \rightarrow 11$ (multiplication before addition)
- `"Hello" + " " + "World" \rightarrow "Hello World"` (concatenation)
- Parentheses override precedence

Afternoon Session Script

13:00 - 14:00 | EXERCISE: Spot the TypeError

[SAY] "Now for the tricky part. Sometimes we mix wrong types. Let's find the errors."

[PROJECT/WRITE]

Error 1:

```
"Hello" * 3 # Works - repeats string  
"Hello" + 3 # TypeError!
```

Why? Can't add string and int.

Error 2:

```
5 + 3.14 # Works (float result)  
5 + "3"  # TypeError!
```

Why? Can't add int and str.

Error 3:

```
[1,2] + [3,4] # Works  
[1,2] + 3     # TypeError!
```

Why? Can't add list and int.

Error 4:

```
10 / 0 # ZeroDivisionError!
```

Why? Can't divide by zero.

[SAY] "Python is strict about types. Let's practice fixing these errors."

[HANDOUT - Worksheet]

Exercise: Fix the TypeError

```
# 1. Fix this:  
result = 5 + "3"  
  
# 2. Fix this:
```

```
result = [1, 2] + 3
```

```
# 3. Fix this:  
result = "Hello" + 5
```

```
# 4. Fix this:  
result = 10 / 0
```

[SOLUTIONS]

```
# 1. Convert string to int  
result = 5 + int("3") # 8  
# Or convert int to string  
result = str(5) + "3" # "53"
```

```
# 2. Convert int to list  
result = [1, 2] + [3] # [1, 2, 3]
```

```
# 3. Convert int to string  
result = "Hello" + str(5) # "Hello5"
```

```
# 4. Check for zero  
if b != 0:  
    result = 10 / 0  
else:  
    result = "Cannot divide by zero"
```

14:00 - 15:00 | BUILD: Complete Python Data Map

[SAY] "Let's build a complete data map in Python for a real scenario."

[PROJECT/WRITE]

Scenario: Student Management System

Group Exercise (30 minutes):

- Teams of 3-4
- Each team writes Python code to represent the data
- Include variables, data types, and operations

What they should create:

Data Type	Variables (Python)	Operations
int	age , grade , score	+ , - , * , / , >

Data Type	Variables (Python)	Operations
float	average , gpa	+ , - , * , /
str	name , course , email	+ , len() , .upper()
bool	is_passing , is_enrolled	and , or , not
list	scores , courses	len() , [] , +
dict	student , course_info	["key"] , .get()
None	missing_data	is None

Team Presentations (5 minutes each):

- Each team shares their code
- Facilitator adds feedback

15:00 - 15:30 | DAY 1 WRAP-UP

[SAY] "Let's recap Day 1."

[WRITE ON WHITEBOARD - Fill in with group]

1. Python Data **Types**: ____, ____, ____, ____, ____, ____, ____
2. Variable = ____ that points **to** a ____
3. FP **Rule**: Variables are ____ (never change)
4. Operators = ____ we can **perform on** data
5. TypeError = Mixing ____ **types**

Answers:

1. int, float, str, bool, list, dict, None
2. Name, Value
3. Immutable
4. Actions
5. Different

[SAY] "One question: **What's one thing that surprised you today?** "

(Collect answers)

[CLOSING STATEMENT] "Today we learned **WHAT** data is and **WHAT** we can do with it in Python. Tomorrow we'll learn **HOW** to make decisions (conditions) and **ASK** questions (expressions)."

[HOMEWORK] "Write a Python script that:

1. Creates variables for your name, age, and favorite color
2. Uses `type()` to check each
3. Prints the results"

Day 1 Facilitator Cheat Sheet

Mistake	How to Fix
Confusing <code>"123"</code> (str) with <code>123</code> (int)	"Quotes = text. No quotes = number."
Using <code>=</code> instead of <code>==</code>	"= is assignment. == is comparison."
Trying to add string and int	"Convert first with <code>int()</code> or <code>str()</code> ."
Case sensitivity errors	"Python is case-sensitive. <code>Name</code> $\neq$ <code>name</code> ."
Forgetting quotes for strings	"Strings need quotes: <code>"Hello"</code> "

Day 2: Python Facilitator Guide

Python Expressions, Statements & Conditions

Session Plan Overview

Time	Activity	Duration
09:00-09:15	Day 1 Recap	15 min
09:15-09:45	Python Expressions Theory	30 min
09:45-10:15	Python Statements Theory	30 min
10:15-10:45	EXERCISE: Expression or Statement?	30 min
10:45-11:15	Python Conditions Theory	30 min
11:15-11:45	EXERCISE: Build Decision Trees in Python	30 min
11:45-12:15	EXERCISE: If/Elif/Else Practice	30 min
12:15-13:00	LUNCH	45 min
13:00-14:00	EXERCISE: Spot the Statement	60 min
14:00-15:00	BUILD: Complete Python Decision System	60 min

Time	Activity	Duration
15:00-15:30	Day 2 Wrap-Up	30 min

Morning Session Script

09:00 - 09:15 | Day 1 Recap

[SAY] "Quick recap from yesterday. **Name one Python data type.** "

(Call on random participants)

"What does `type()` do? "

(Answer: Returns the type of a variable)

"What's a `TypeError`? "

(Answer: Mixing wrong types)

"What's the FP rule for variables? "

(Answer: Never change them - create new ones)

"Excellent. Now we build on that."

09:15 - 09:45 | Python Expressions Theory

[SAY] "Now we enter the CORE of Python programming. **Everything in programming is about asking questions and getting answers.** "

[WRITE ON WHITEBOARD]

EXPRESSION = A combination of **values**, **variables**, **and** operators
that produces a **VALUE**.

Examples:

`5 + 3` → 8

`"Hello" + "!"` → `"Hello!"`

`age >= 18` → `True` or `False`

`len(name)` → Number of **characters**

[SAY] "Every expression gives you something back. You ask a question, you get an answer. No side effects."

[DEMONSTRATE LIVE]

```
# Expressions
print(5 + 3)           # 8
print("Hello" + "!")  # "Hello!"
print(age >= 18)       # True or False
print(len("Hello"))    # 5

# Expressions can be nested
print((5 + 3) * 2)     # 16
print(len("Hello" + " " + "World")) # 11
```

[SAY] "Notice: **Every expression gives you something back.** You can use the result immediately."

09:45 - 10:15 | Python Statements Theory

[SAY] "Now let's talk about statements. Statements are commands. They don't ask questions. They TELL Python to do something."

[WRITE ON WHITEBOARD]

STATEMENTS = Commands **that** perform actions.

Examples:

```
x = 5                # Assignment statement
print("Hello")       # Print statement
if x > 10:            # Condition statement
    print("x is big")
for item in list:     # Loop statement
    print(item)
```

[SAY] "Statements have SIDE EFFECTS. They change things outside the program."

[DEMONSTRATE]

```
# This is a statement
x = 5                # Changes the program's memory

# This is an expression
x + 3                # Produces 8, but doesn't change anything

# This is a statement
print("Hello")      # Prints to screen, returns None

# This is a statement
if x > 10:
    print("Big")     # Changes program flow
```

[SAY] "In functional programming, we use expressions for ALL logic. We use statements ONLY for input/output and control flow."

[WRITE ON WHITEBOARD]

FP Rule: All logic = EXPRESSIONS
Only final output = STATEMENTS

10:15 - 10:45 | EXERCISE: Expression or Statement?

[SAY] "Let's practice. I'll show you Python code. You tell me if it's an expression (E) or statement (S)."

[PROJECT - One at a time]

Code	E or S?	Why?
5 + 3	E	Produces a value
print("Hello")	S	Performs an action
age = 25	S	Assignment statement
name.upper()	E	Returns a string
if age > 18:	S	Control flow statement
len(scores)	E	Returns a number
for x in list:	S	Loop statement
x > y	E	Returns True/False

[DEBRIEF]

- "What's the key difference?" → Expressions produce values; statements perform actions
- "When do we use statements?" → For assignment, conditions, loops, printing

[SAY] "In FP, expressions are your workhorses. Statements are just the scaffolding."

10:45 - 11:15 | Python Conditions Theory

[SAY] "Now let's add DECISIONS. Computers aren't smart - they just follow rules. But we can make them APPEAR smart by using conditions."

[WRITE ON WHITEBOARD]

CONDITION = A decision point **in** Python.

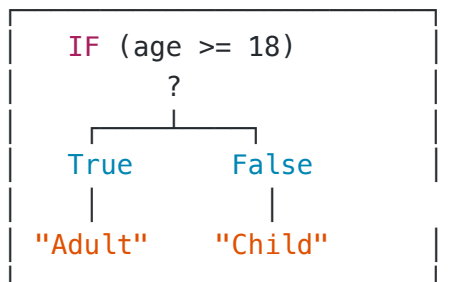
Structure:

```
if condition:
    # code if True
elif condition2:
    # code if condition2 True
else:
    # code if all False
```

Example:

```
if age >= 18:
    print("Adult")
else:
    print("Child")
```

[DRAW ON WHITEBOARD]



[DEMONSTRATE LIVE]

```
# Simple if
age = 25
if age >= 18:
    print("Adult")
```

```
# if-else
age = 16
if age >= 18:
    print("Adult")
else:
    print("Child")
```

```
# if-elif-else
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
```

```
else:
    grade = "F"
print(grade) # "B"

# Multiple conditions with and/or
age = 25
is_student = True
if age >= 18 and is_student:
    print("Student discount")
elif age >= 18:
    print("Regular price")
```

11:15 - 11:45 | EXERCISE: Build Decision Trees in Python

[SAY] "Your turn. Write Python code for these scenarios."

Scenario 1: Voting Eligibility

```
age = int(input("Enter age: "))
if age >= 18:
    print("Eligible to vote")
else:
    print("Not eligible to vote")
```

Scenario 2: Grading System

```
score = int(input("Enter score: "))
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"
print(f"Grade: {grade}")
```

Scenario 3: Weather App

```
temp = float(input("Enter temperature: "))
if temp > 30:
    print("Hot")
elif temp > 20:
    print("Warm")
elif temp > 10:
    print("Cool")
```

```
else:  
    print("Cold")
```

[GIVE 10 MINUTES]

[DEBRIEF] Check each scenario:

- Scenario 1: Correct use of if/else
- Scenario 2: Correct use of if/elif/else
- Scenario 3: Correct temperature ranges

11:45 - 12:15 | EXERCISE: If/Elif/Else Practice

[SAY] "Let's write a more complex condition."

[PROJECT/WRITE]

Cinema Ticket Pricing:

- Regular **ticket**: \$10
- Student **ticket**: \$8 (must be 18+ AND have student ID)
- Child **ticket**: \$5 (must be under 12)
- Senior **ticket**: \$7 (must be 65+)

```
age = int(input("Enter age: "))  
is_student = input("Are you a student? (yes/no): ").lower() == "yes"  
  
if age < 12:  
    price = 5  
elif age >= 65:  
    price = 7  
elif age >= 18 and is_student:  
    price = 8  
else:  
    price = 10  
  
print(f"Ticket price: ${price}")
```

[GIVE 10 MINUTES]

Afternoon Session Script

13:00 - 14:00 | EXERCISE: Spot the Statement

[SAY] "Now let's identify statements vs expressions in Python code."

[PROJECT/WRITE]

```
# Code to calculate total with discount
total = 0
for price in prices:
    total = total + price
discount = total * 0.1
final = total - discount
print(final)
```

Statements: `total = 0` , `for price in prices:` , `total = total + price` , `print(final)`

Expressions: `total * 0.1` , `total - discount`

[SAY] "Now let's REWRITE this in FP style. Only the final print should be a statement. "

FP Version:

```
# All logic is expressions
prices = [10, 20, 30]
total = sum(prices)           # Expression
discount = total * 0.1       # Expression
final = total - discount     # Expression
print(final)                 # Statement (only at end)
```

[SAY] "Notice: We replaced the for loop with `sum()` . We used expressions everywhere. Only the final output is a statement."

14:00 - 15:00 | BUILD: Complete Python Decision System

[SAY] "Now let's build a COMPLETE Python program using everything we've learned."

[PROJECT/WRITE]

Scenario: Cinema Ticket Pricing System

Requirements:

1. Ask `for` age
2. Ask `if` student (`yes/no`)
3. Calculate ticket price
4. Print final price

Group Exercise (30 minutes):

- Teams of 3-4
- Write complete Python program
- Include:
 - Input using `input()`
 - Conditions using `if/elif/else`
 - Output using `print()`
 - Error handling (`try/except`)

[FACILITATOR SOLUTION]

```

"""
Cinema Ticket Pricing System
"""

def get_ticket_price(age, is_student):
    """Calculate ticket price based on age and student status"""
    if age < 12:
        return 5
    elif age >= 65:
        return 7
    elif age >= 18 and is_student:
        return 8
    else:
        return 10

def main():
    try:
        # Input
        age = int(input("Enter your age: "))
        student_input = input("Are you a student? (yes/no): ")
        is_student = student_input.lower() == "yes"

        # Process
        price = get_ticket_price(age, is_student)

        # Output
        print(f"Your ticket price is: ${price}")

    except ValueError:
        print("Invalid input. Please enter a number for age.")

if __name__ == "__main__":
    main()

```

Team Presentations (5 minutes each):

- Each team shares their solution
- Discuss different approaches

15:00 - 15:30 | DAY 2 WRAP-UP

[SAY] "Let's recap Day 2."

[WRITE ON WHITEBOARD]

1. Expression = ____ that produces a ____
2. Statement = ____ that performs an ____
3. FP Rule: All logic = ____; Only output = ____
4. Condition = ____ using `if/elif/else`
5. Python decides which path `to` take based on ____

Answers:

1. Code, Value
2. Code, Action
3. Expressions, Statement
4. Decision
5. True/False

[SAY] "One question: What's the biggest difference between an expression and a statement in Python? "

(Answer: Expression gives you a value; statement performs an action)

[CLOSING STATEMENT] "Today we learned how to ASK questions (expressions), GIVE commands (statements), and MAKE DECISIONS (conditions). Tomorrow we'll learn how to REPEAT things (loops) and ORGANIZE data (lists and dictionaries)."

[HOMEWORK] "Write a Python program that:

1. Asks for a score
2. Prints the grade (A, B, C, D, F)
3. Uses `if/elif/else`"

Day 2 Facilitator Cheat Sheet

Mistake	How to Fix
Using <code>=</code> instead of <code>==</code> in conditions	" <code>=</code> is assignment. <code>==</code> is comparison."
Forgetting colon <code>:</code> after <code>if/elif/else</code>	"Every condition needs a colon."

Mistake	How to Fix
Wrong indentation	"Python uses indentation to show blocks."
Forgetting <code>else</code>	"What if the condition is false? Always handle."
Using <code>if</code> instead of <code>elif</code>	"elif checks multiple conditions. if creates nested blocks."

Day 3: Python Facilitator Guide

Python Loops, Lists & Data Structures

Session Plan Overview

Time	Activity	Duration
09:00-09:15	Day 2 Recap	15 min
09:15-09:45	Python Loops Theory	30 min
09:45-10:15		